
Hetja

City-scale infrastructure for keeping street dogs alive — a technical guide for investors and their technical advisors.

Subject	github.com/jabezcharles420/hetja — private evaluation copy
Prepared	15 August 2026
Basis	Direct source audit, live production measurement, and full test-suite execution against the production database image
Commit	d8f8e1e (deployed) · fix/ledger-chain-seq pending
Status	Live and serving. Pre-revenue. Four days of version-control history.

This document was produced by reading the repository and measuring the running system, not by interviewing the team. Every quantitative claim is either measured (and marked as such) or cited to a file. Where the repository's own documentation contradicts its code, the code is reported and the contradiction is flagged. **No financial, market, traction or team information was available to the author**, and none is asserted here.

Contents

1	How to read this document
2	Executive summary
3	The problem, and what has been built
4	The four surfaces, and what each one is for
5	Architecture and data model
6	The fifteen invariants — the actual intellectual property
7	Privacy, security and Indian regulatory exposure
8	Engineering quality: an independent audit and what it found
9	Live production verification
10	Cost structure and the scaling ceiling
11	Risk register
12	What investment would actually buy
A	Appendix: measured repository metrics
B	Appendix: glossary for non-specialists

1 • How to read this document

You asked for technical depth without being written down to. This section exists so that the depth is usable rather than decorative.

The document is layered. Each numbered section opens with a plain-language paragraph stating what the section is about and why it matters commercially. The detail that follows is aimed at a technical advisor doing due diligence. You can read only the opening paragraphs and the tables and come away with an accurate picture; the rest is there so that someone you trust can check my work.

Three phrases you will meet constantly

An invariant is a rule the system promises never to break — for example, "a stranger who scans a collar can never see the dog's exact location." The commercial question about any such promise is always the same: *is it enforced by a machine, or is it just written down?* Section 6 answers that for all fifteen, one at a time, and the distinction is the single most important thing in this document.

A gate is an automated check that refuses to let code reach production if a rule is broken. Gates are how a promise becomes mechanical rather than cultural. A gate that cannot fail is worse than no gate, because it produces false confidence — and I found two of those, described in section 8.

The ledger is a tamper-evident record of veterinary treatment. Each record is cryptographically linked to the one before it, so altering an old record breaks every link after it and is detectable. Section 7 explains precisely what that does and does not prove.

ON MY INDEPENDENCE

I was asked to improve this codebase, and I did: three changesets, two already deployed to production. That makes me a contributor, not a neutral party. I have tried to correct for it by reporting defects I introduced myself alongside those I found, and by marking every claim as measured, cited, or uncertain. Where I judged something and could be wrong, I have said so rather than smoothing it over.

2 · Executive summary

21.9k

LINES OF SOURCE

9.5k

LINES OF TESTS

445

TESTS PASSING

4 days

OF GIT HISTORY

What it is. Hetja is working infrastructure for keeping street dogs alive in Mumbai: a QR-coded collar tag, a public web page any passer-by can load without installing anything, a geofenced emergency network that pages nearby responders, and a tamper-evident medical ledger. It is live — the production system answered requests while this document was being written — and it is open source by deliberate policy.

What is genuinely unusual about it. Two things. First, the engineering discipline is far above what its age would predict: fifteen written invariants, seven automated gates enforcing them, a hard 40 KB budget on the page a stranger loads, and 9,500 lines of tests against 21,900 lines of source. Second, the reasoning is written down. The codebase explains *why* decisions were made, including its own past mistakes, at a density I rarely see in mature commercial code. That is a real asset: it is what let an outside auditor become productive in hours rather than weeks.

What an investor must weigh against that. The version-control history is four days old, 59 of 109 commits are AI co-authored, and the system has no measured traction of any kind that I could find. The discipline is real but young, and young discipline has not yet been tested by the things that break systems: concurrency, a second machine, an unattended deploy at 3 a.m. My audit found exactly that pattern — the code was well-reasoned and well-documented, and the defects clustered precisely where reasoning cannot substitute for execution.

What the audit found

I audited the codebase against its own stated invariants and found **fourteen defects**, every one of them silent: nothing crashed, no test went red, and in three cases an automated gate actively reported a property intact while it was broken. The three that matter most:

DEFECT	CONSEQUENCE	STATUS
The job queue could not retry	A single failing job halted the 8-minute emergency escalation for the entire system, silently. The retry counter was rolled back by the same transaction that was supposed to record it, making the give-up branch unreachable code.	FIXED & DEPLOYED
The emergency ledger forked under load	The lock meant to prevent this did not: 16 concurrent writes inverted 42.5% of ordering pairs and created 3 forks. The public verification endpoint then reported <i>TAMPERED</i> on data nobody had touched.	FIXED, AWAITING MERGE
Two gates could not fail	One would pass a configuration with the life-safety cache rule deleted. The other would let <code>DROP TABLE medical_records</code> through to an unattended production migration.	FIXED & DEPLOYED

I regard finding these as evidence *for* the project rather than against it. They were findable because the invariants were written down precisely enough to test against — most codebases cannot be audited this way at all. But they were present, which means the promises were not yet mechanically true.

Three gaps that outweigh every defect

The audit above concerns correctness. Separately, tracing the product end to end found three structural gaps that a technical diligence process would surface immediately, and that matter more than any bug:

GAP	WHAT IT MEANS
There is no way to enrol a dog	No tagger application exists, and no endpoint to create a dog or bind a collar is registered. Dogs can currently be added only by a seed script or hand-written SQL. The system that registers street dogs cannot yet register a street dog.
The emergency loop does not close	Only reports marked <i>critical</i> notify anyone — and the app's emergency form defaults to <i>serious</i> , which notifies nobody. The push notification links to a screen that does not exist. The 8-minute escalation writes database rows addressed to channels with no sender. See §3.4.
The ledger is not yet evidence against its operator	The daily fingerprint that makes the chain auditable is computed and stored but published nowhere . It currently proves integrity against an intruder, not against Hetja. See §7.3.

None of this is concealed — the repository documents most of it against itself, which is a genuine mark in its favour. But an investor should hold an accurate picture of what is deployed: **a working identification and phone-directory system for dogs that cannot yet be enrolled, with a correctly-modelled emergency case that in its default configuration pages nobody.**

THE HONEST HEADLINE

This is an unusually well-architected and exceptionally well-documented four-day-old system, live but with no measured usage. The engineering judgement on display is real and well above the norm for its age. Ten of its fifteen stated invariants are mechanically enforced.

The risk is not the code. It is that the *mission-critical path is incomplete* — no enrolment, no closed emergency loop — while the operational base is thin: one shared 2 GB server in the wrong continent, backups written to the disk they back up, no restore ever performed, and no rate limiting on the endpoint that sends email. Those are weeks of work, not quarters. But they are the work, and they are not features.

3 · The problem, and what has been built

Mumbai has a large street-dog population, no register of which animal is which, and no mechanism for a bystander to summon help for an injured one. Hetja is an attempt to build that register and that mechanism as public infrastructure rather than as a company asset.

3.1 · The mechanism in one paragraph

A dog wears a collar carrying a QR code. Anyone who finds the dog points a phone camera at it and lands on a web page — **no app, no account, no login** — showing the dog's identity, vaccination status, and one large button reading "this dog needs help". Pressing it opens an emergency case, surfaces tappable phone numbers for nearby veterinary and NGO providers, and is designed to wake nearby registered responders. A separate signed-in application lets volunteer feeders log meals and photographs, which accrues a trust score. Veterinary treatment is written into an append-only, cryptographically chained ledger.

3.2 · Why it exists

The origin is documented in the repository and is unusually specific. The maintainer, as a child leaving an abusive home, walked three kilometres through rain; a stray he had occasionally fed followed him the whole way and barked down other dogs on the road. He learned years later that it had been poisoned. The repository records the consequence plainly:

"I went inside. I never said goodbye. I never hugged it.
I never thanked it. I never gave it a name."
— docs/design/MEMORIAL-CONTENT.md:66

This matters commercially for one reason: the design constraints are traceable to it, and they are unusually disciplined as a result. Collar codes are random rather than sequential, and coordinates are deliberately coarsened, because — in the repository's words — *"a georeferenced list of every stray in a city is, in one political climate, a tool for protection. In another, it is a targeting list."* The project's governing rule is stated as: **"The system is allowed to know less than it wants to. It is not allowed to *claim* more than it knows."**

3.3 · What is genuinely built, and what is not

This is the most important table in the document for judging maturity. It is derived from reading the code, not the documentation — in several places the two disagree, and where they do, the code is reported.

CAPABILITY	STATE	EVIDENCE
Stranger scans a collar → sees the dog, gets phone numbers	WORKS	End to end, deployed, measured live. HMAC signature on the code is verified server-side; location is coarsened before it is returned.
Feeder logs a feed, offline-capable	WORKS	Email code login, in-browser photo compression with GPS metadata stripped, queued in the browser and replayed idempotently when signal returns.
Medical ledger — hash chain, Merkle proofs, append-only	API ONLY	Cryptographically rigorous and enforced by the database in two independent ways. There is no veterinary user interface anywhere — it can only be written to via the API.

CAPABILITY	STATE	EVIDENCE
SOS → responders are woken	HALF-BUILT	See §3.4. This is the mission statement, and it is the least complete part of the system.
Enrolling a dog / binding a collar	ABSENT	No mechanism exists. The tagger portal is not built, and no route to create a dog is registered. Dogs can currently only be added by a seed script or by hand-written SQL.
AI photo validation	STUB	The detector returns "no dog" unconditionally and is labelled <code>stub-v0</code> . Nothing schedules it.
First-aid guidance on the collar page	BUILT, DISABLED	Deliberately switched off pending sign-off from a practising vet. The source carries the comment <code>*** DO NOT SET FIRST_AID_ENABLED TO true ***</code> .
iOS emergency notifications	BLOCKED	Requires a native wrapper application. That directory exists and is empty.

3.4 · The gap that matters most

THE EMERGENCY LOOP DOES NOT CLOSE

The project's mission is that an injured dog gets help. Tracing that path through the code, four independent breaks appear:

- **Only cases marked *critical* notify anyone.** Cases marked minor or serious notify nobody at all — and the emergency form in the feeder app **defaults to "serious"**. The default emergency report therefore wakes no one.
- **The notification links to a screen that does not exist.** A responder who receives a push and taps it lands on a dead route. The acknowledgement API is correctly built and well-designed; nothing in the product calls it.
- **The 8-minute escalation terminates in a database row.** It writes notification records addressed to SMS and to the municipal body. No sender exists for either channel.
- **iOS responders are unreachable** without the unbuilt native wrapper.

None of this is hidden — the repository documents most of it against itself. But an investor should understand precisely what is deployed: *a working identification and phone-directory system*, plus a correctly-modelled emergency case that, in its default configuration, currently pages nobody.

4 · The four surfaces

Four separately-deployed applications, deliberately isolated so that a bad release of one cannot take down the others. The isolation is a safety decision, not an architectural fashion.

SURFACE	TECHNOLOGY	PURPOSE AND NOTABLE CONSTRAINT
apps/scan	Plain TypeScript, 2 runtime dependencies	The page a stranger loads. Held to a hard 40 KB gzipped budget enforced by CI . Deployed separately so an unrelated release cannot break it. Measured live at 11.9 KB.
apps/web	Next.js 14, 11 dependencies	The signed-in feeder application plus public content, delivered as an installable offline-capable web app so no app store is required.
apps/api	Fastify 5 + schema validation	26 endpoints under <code>/api/v1</code> . Enforces every privacy rule at the boundary.
apps/worker	Node, PostgreSQL-backed queue	Emergency escalation, notifications, retention, daily ledger anchoring. No Redis — a deliberate rule, driven by the memory budget.

4.1 · Why the collar page has a 40 KB budget

This is the clearest example of the project's engineering character, so it is worth quoting the reasoning directly:

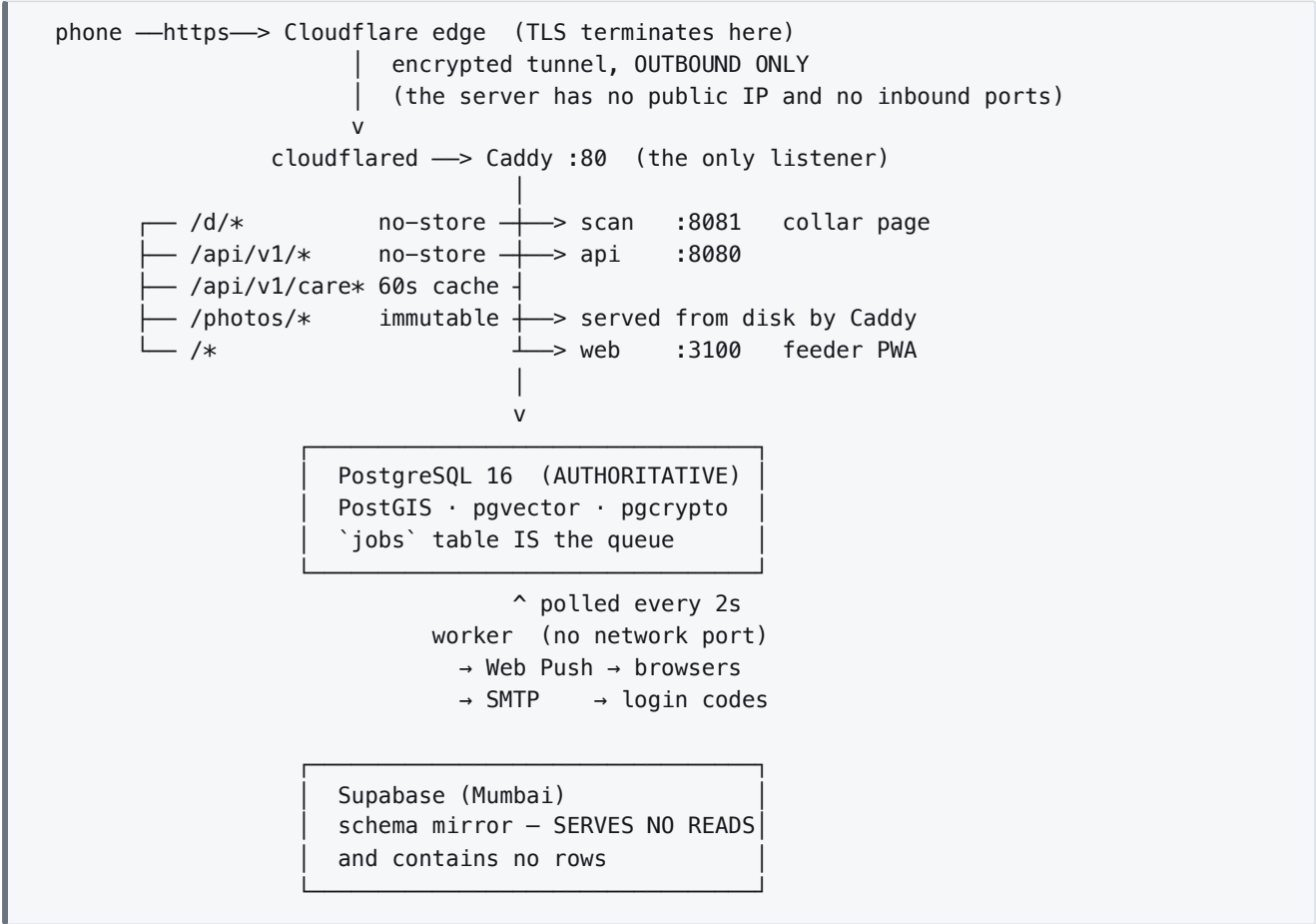
```
"A framework runtime alone would blow the budget before a single line
of the app's own code ran, on exactly the lowest-bandwidth,
highest-urgency path in the system."           — docs/INVARIANTS.md
```

The person loading that page is standing on a street over an injured animal, on a congested mobile network. Every kilobyte is a second. The budget is enforced by an automated gate that fails the build — and during this audit I found and fixed a flaw in that gate, described in §8.

5 · Architecture and data model

Everything runs on one small virtual machine behind an outbound-only tunnel. That is both the system's most elegant property and its single largest risk.

5.1 · Topology



All four application services bind to loopback only. Even if a port were accidentally exposed, they could not be reached except through Caddy.

TWO FACTS AN INVESTOR SHOULD NOT MISS

The server is physically in Europe, serving users in Mumbai. A move to India is referenced in four places in the repository and has not been executed.

The machine is shared with unrelated workloads. The repository documents tuning it to host a fleet of other processes alongside production. It also cannot swap — swap is blocked by the container's security policy.

5.2 · The data model

Nineteen tables across twenty migrations. The structurally interesting ones:

TABLE	ROLE
dogs	The register. Carries a coarse last-seen location as a geographic type, and a reserved 768-dimension vector column for future photo-based re-identification (unused today).

TABLE	ROLE
scans	Every interaction with a collar. Carries a client-generated unique identifier which is what makes offline replay safe — a phone retrying a queued feed produces exactly one row, not one per retry.
medical_records	The ledger. Append-only, enforced by the database in two independent ways. Each row links cryptographically to its predecessor.
feeders	Volunteers. Identity is stored only as a keyed hash — never the raw email — under a secret held outside the database.
jobs	The queue itself. No Redis; PostgreSQL's row-locking provides the same guarantees on a machine that cannot afford a second service.

The geographic indexing is worth one note for a technical reviewer: the index serving emergency responder lookup *encodes the eligibility rule in the index itself* — it indexes only feeders who have opted in and who exceed a trust threshold. That is a sophisticated piece of database design, and it is representative of the codebase's general level.

5.3 • Deployment

Pushing to the main branch triggers three stages. Each guarantees something different:

1. **Gate** — type checking, the full test suite against a real database, and seven independent gates. Nothing proceeds if any fails.
2. **Migrate** — database changes are screened for destructive operations, then applied.
3. **Deploy** — the front-end is built on the CI machine (it needs about 1 GB and has previously killed the live services when built on the server), shipped, and the server rebuilds the back-end from the same commit. It then verifies that the deployed commit matches the intended one.

WHAT DOES NOT ROLL BACK

Code rolls back automatically; **database schema does not**. This is only safe because the destructive-change gate keeps unattended migrations additive — which is precisely why the flaw I found in that gate (§8.2) mattered more than its size suggests.

Separately, the health checks after deployment verify the internal services but **not the tunnel, DNS or TLS**. A total outage visible to every user would pass every check.

6 · The fifteen invariants — the actual intellectual property

Hetja's distinctive asset is not its code, which is replaceable. It is a written list of fifteen rules the system promises never to break, several of which are enforced by machines rather than by intention. This section establishes which ones are real.

The commercial question is simple. A promise enforced by a database constraint survives staff turnover, a rushed release, and a careless contractor. A promise that exists only in a document survives none of those. The classification below was produced by tracing each rule to its enforcement point, and **deliberately biased downward**: where a mechanism exists but sits on a code path nothing invokes, it is classified as documentation, because a dead code path is not a commitment.

#	THE RULE	ENFORCEMENT	BASIS
5	Offline replay is idempotent — a phone retrying a queued feed produces one row, not one per retry	DATABASE	A unique index. The strongest of the fifteen — a constraint, not a convention.
8	Medical records accept inserts only — no edits, no deletions, no truncation	DATABASE	Privilege revocation plus an all-roles trigger. <i>Limit</i> : edits are blocked by privilege, so a superuser on the machine can still rewrite rows.
1	Collar codes are random, never sequential	CODE	Cryptographic random source, with tests. <i>Note</i> : the check character is ineffective (§8.4).
2	Public reads never return exact coordinates	CODE	Coarsening applied at the response boundary, plus a CI grep as backstop. See §7.2 for what "coarse" means in metres.
3	Contact details stored only as a keyed hash, never in the clear	CODE	Keyed HMAC under a secret pepper. <i>Limits</i> : a development default exists and is only mandatory in production; published clinic phone numbers are a deliberate, documented exception.
4	Offline conflicts resolve on capture time, not arrival time	CODE	Enforced atomically inside the update statement.
7	Anonymous emergency reports require an attested device, capped 2/day and 5/week	CODE	Real and tested. <i>Limits</i> : the cap is a count-then-insert with no lock, so concurrent requests can overshoot; the windows reset on calendar boundaries rather than rolling.
9	The ledger is hash-chained with unambiguous hash inputs	CODE	Correctly implemented and tested. <i>Limit</i> : nothing in the database requires a row to actually link to its predecessor — a direct insert bypassing the application writes an unchained row.
12	Every documented query is checked against the real schema	CI GATE	Blocking, runs against a real database. Covers documented queries only.
13	The collar page stays under 40 KB compressed	CI GATE	Blocking. Now walks the whole build output, so new code counts by default (a fix from this audit).
6	Rate limits are per account or device, never per IP address	DOC ONLY	Nothing enforces the negative, and there is no general rate limiter at all . True today only because the two limits that exist happen to be keyed correctly.

#	THE RULE	ENFORCEMENT	BASIS
10	Publish the ledger head daily, somewhere the operator does not control	DOC ONLY	Computation and signing are built; publication is not . See §7.3 — this is the invariant that makes the ledger evidence.
11	Erasure requests delete personal data while the chain stays valid	DOC ONLY	Design intent. The documented procedure would fail if executed today — see §7.4.
14	AI validation flags for review; never silently rejects	DOC ONLY	Nothing anywhere schedules the validation job . Marked complete in the project's own table; it is not.
15	Provisional volunteers auto-pause after three serial rejections	DOC ONLY	The logic is real and tested but is never called from any route , and its input is never produced. Marked complete; it is not.

6.1 · Reading the scorecard

2 DATABASE-ENFORCED	6 CODE-ENFORCED, TESTED	2 BLOCKING CI GATES	5 DOCUMENTED ONLY
-------------------------------	--------------------------------------	-------------------------------	-----------------------------

Ten of fifteen are mechanically real. For a system this young that is a genuinely strong result, and it is the single best evidence that the engineering culture is substantive rather than performative.

THE FINDING THAT MATTERS

Of the five that are documentation only, **the project self-reports two** (10 and 11) and marks the other three — 6, 14 and 15 — as complete in its own table. They are not. Invariant 14 in particular exhibits precisely the failure the project already documented and post-mortemed for invariant 10: a rule marked done against a background job that *nothing ever schedules*.

This should not be read as dishonesty. The repository's own post-mortem of that earlier case is the most self-critical document I found in it. It should be read as an **immature verification culture**: the team writes rules well, implements them well, and does not yet consistently test that the implementation is reachable. That is a fixable process gap, and the three affected rules are the ones to fix first.

7 · Privacy, security and Indian regulatory exposure

The project treats location privacy as a safety property rather than a compliance chore, for a stated reason: a register of where every street dog sleeps is protective in one political climate and a targeting list in another.

7.1 · What personal data exists

Deliberately little. Anonymous reporters supply *no identity at all* — only a pseudonymous device identifier. Volunteers supply an email address, which is never stored: only a keyed hash of it. Photographs have their embedded metadata, including GPS coordinates, stripped **server-side before the record is created** — the browser-side stripping is correctly treated as a convenience rather than a control.

Two exposures a technical reviewer should note, because they are not covered by the hashing:

- **Volunteer display names are stored in the clear alongside an exact location point.** The hash protects the login channel, not the person. Anyone with database access has a named individual and their precise position. The project evaluated encrypting those coordinates and rejected it on sound technical grounds — geographic queries cannot run against ciphertext — but the residual exposure is real.
- **Photograph pixels are unaddressed.** Metadata is stripped rigorously; a doorway, a shopfront or a nameplate visible in the image is published unredacted.

7.2 · Location coarsening, in metres

The published precision is worth quantifying exactly, because the project's own documentation describes it imprecisely.

MEASURE	VALUE
Coarsening method	Truncation to two decimal degrees — despite the function's name, no administrative ward polygon is involved
Cell size at Mumbai	1,113 m north–south × 1,052 m east–west ($\approx 1.17 \text{ km}^2$)
Worst-case displacement	$\approx$ 1,531 m — truncation places the published point at the cell's south-west corner, not its centre
Also published	The administrative ward label, verbatim, alongside the coarsened point
Heatmap aggregation	$\approx 189 \text{ m}$ cells internally, then re-quantised to the same $\sim 1.1 \text{ km}$ lattice on output
k-anonymity floor	k = 3 distinct active dogs per cell, applied at the finer grain — stricter than required, which is the right direction
Storage	Exact coordinates are stored at rest. Coarsening is applied when shaping the response, not when writing.

One literal deviation worth recording: the invariant's prose specifies cells of 500 m or larger, and the heatmap aggregates at $\sim 189 \text{ m}$ before re-quantising. The published output is coarser than the rule requires, so the deviation is technical rather than harmful — but the function that actually implements the documented 500 m behaviour is dead code, called only by its own test.

7.3 · What the tamper-evident ledger actually proves

The cryptography is competently done: each record's fingerprint incorporates its predecessor's, with length-prefixed inputs so two different records cannot collide by construction; each write also stores a Merkle root

permitting an efficient inclusion proof; and the proof endpoint is deliberately **unauthenticated**, on the correct reasoning that a proof requiring a credential the operator can revoke would be worthless against that operator.

THE LIMIT, STATED PRECISELY

The daily head is computed, optionally signed, stored — and **published nowhere**. The field intended to hold its public location is written as an empty string.

Consequently: **the ledger is today tamper-evident against an attacker who reaches the database, but not against Hetja itself**. An operator who rewrote history could recompute the chain and re-anchor it, and no external party holds an earlier value to contradict them. Signing is off by default, and no public-key document is served, so a signature currently verifies against nothing.

The gap is a small amount of work — publish the daily head somewhere third parties can retain it. Until that is done, the ledger's marketable claim should be stated as integrity against intrusion, not independent auditability.

7.4 • India's DPDP Act: an unresolved tension

The project has thought about this harder than most at its stage. Consent version and minor status are recorded as first-class columns; the design principle is that the immutable ledger holds no personal data, so an erasure request can delete the identity record while the chain stands.

Four things are unresolved, and I will not resolve them here either:

1. **A veterinarian is a natural person.** Medical records carry a vet identifier, and vets are linked to personal accounts. An erasure request *from a vet* touches rows that are undeletable by construction. The repository never discusses this case.
2. **Free-text clinical fields are unconstrained.** "No personal data in the ledger" is a convention about what callers submit, not a mechanism that prevents it.
3. **The documented erasure procedure would fail if executed today.** Eight foreign keys reference the volunteer record with no cascade behaviour defined; the deletion the runbook prescribes would error on any referencing row, and no code implements the cascade.
4. **No request-handling machinery exists** — no intake route, no audit log, no retention job.

For an investor: this is a *plausible architecture* for reconciling immutable veterinary evidence with a statutory erasure right, with an unexamined case at its centre and no operational implementation. It is a diligence item, not a blocker, but it should be costed as legal work rather than assumed.

7.5 • Security posture

AREA	ASSESSMENT
Attack surface	Good. No inbound ports. All services bind to loopback and are reachable only through the proxy. TLS terminates at the CDN edge.
Injection	Clean. Every dynamic value is parameterised; the only string interpolation is of module-level integer constants.
Cryptographic comparisons	Now constant-time throughout, following this audit.
Rate limiting	Absent entirely. The login endpoint is uncapped and sends email synchronously.

AREA	ASSESSMENT
Privilege	All four services run as root with minimal hardening directives. A remote-code-execution flaw in any of them is immediately root, with access to the secret that signs every printed collar.
Secrets	A database password is committed in a public repository and is present in git history.

8 • Engineering quality: an independent audit

This is the section a technical advisor should read first. It is also the section I have the most direct evidence for, because I performed the audit and fixed what it found.

8.1 • Method

I cloned the repository, brought up the exact database image CI uses (`postgis/postgis:16-3.4` plus `pgvector`), applied all migrations, and ran the entire test suite — which required a real PostgreSQL with geospatial and vector extensions, not a mock. I then audited five areas in parallel against the project's own written invariants, and verified each finding by reproducing it rather than by reading.

That distinction matters. Every significant claim below was reproduced against a running system. Where a finding was reported to me but did not survive verification, I rejected it — one such case is documented in §8.5, because how a team handles a wrong-but-plausible bug report says more than the bug count.

8.2 • What was found

Fourteen defects, every one silent. Nothing crashed. No test failed. In three cases an automated gate reported the property intact while it was broken. That pattern is the finding: this is not a sloppy codebase, it is a careful codebase whose careful claims had not been mechanically verified.

Defect 1 — The job queue could not retry, and one bad job stopped all the others

The queue delivers emergency escalation and responder notifications. Its claim, restart, and delete all ran inside *one* database transaction. When a job failed, the rollback that undid the job's partial writes also undid the attempt counter and the lease that recorded the failure. Three consequences, all silent:

- The attempt counter never advanced, so the "give up after 8 tries" branch was **unreachable code** — it could never execute.
- The job kept its original schedule, so the queue re-selected the same failing job every two seconds, forever.
- The failure propagated out and aborted the whole batch, so **one poison job stopped every job queued behind it**.

In plain terms: one emergency-escalation job whose dog record had been deleted would silently halt the 8-minute escalation ladder for the entire city, with a single line in a log as the only evidence.

Fix: split into three transactions — claim, run, settle — with exponential backoff, plus a migration adding a dead-letter column so an exhausted job is parked and visible rather than deleted. Deleting it was not an option: these are life-safety jobs, and a job that vanishes after eight failures is precisely the silent rejection the project's own INVARIANT 14 forbids.

Defect 2 — The medical ledger forked under ordinary concurrency

This is the most serious finding, and the most interesting. The ledger ordered its records by `created_at`. In PostgreSQL that value is the *transaction start* time, stamped when the transaction opens — which happens one network round trip *before* the lock that decides append order is acquired. A transaction can therefore start earlier and append later.

16 concurrent appends through the real endpoint, no contrived timing:

```
inversions:      51 / 120 pairs    (42.5%)
chain forks created: 3
```

42.5% is statistically near-indistinguishable from a random shuffle. The surprise was the forks: the advisory lock, which exists precisely to prevent them, does not — it serialises the *writes* but not the choice of *what to link onto*. Once one pair inverts, every later record links onto a parent that already has a child. The table permits no updates, so this cannot be repaired after the fact.

The user-visible symptom is the worst one available to a tamper-evidence system:

```
db order  [B,A]  verify -> valid=false => TAMPERED
true order [A,B]  verify -> valid=true  => VALID

Same bytes. Same stored hashes. Only the sort order differed.
```

A public endpoint reporting *TAMPERED* on untouched data does not merely fail — it trains whoever watches it to ignore the alarm, which destroys the value of the entire ledger.

WHY THIS ONE IS COMMERCIALLY SIGNIFICANT

The ledger is the project's differentiating claim: that its records are evidence a court or a municipal audit could rely on. An ordering bug that makes clean data report as tampered attacks that claim directly. It is now fixed, and the fix is verified in both directions — the new test fails against the old ordering and passes against the new one — but an investor should note that the flagship feature was, until this week, not delivering its flagship property under load.

Defect 3 — Two automated gates could not fail

Gates are how this project turns written promises into mechanical ones, so a gate that cannot fail is worse than no gate. I tested both by deliberately sabotaging their inputs.

- **The cache-policy gate.** The collar page must never be cached — a stale emergency page is a life-safety bug. I deleted the rule enforcing that and left a comment mentioning it. The gate printed `PASS: /d/*` is no-store and never cached and exited successfully, because it matched the *word* anywhere on a line, including in comments.
- **The destructive-migration gate.** This is the check that stops an unattended deployment from destroying data. It stripped SQL comments with a pattern that also truncated real SQL following a comment marker inside a text value. The result: `INSERT INTO t VALUES ('a -- b'); DROP TABLE medical_records;` passed the gate. Against an append-only table whose loss is unrecoverable.

Both are fixed and both are now negative-tested — I verified the old versions pass the sabotage and the new versions catch it. The destructive-migration gate immediately proved its worth by catching a `DROP INDEX` in *my own* migration, which I then rewrote.

Defects 4–14 — summary

AREA	DEFECT
Offline feed queue	The clock-skew rule rejected any feed queued offline for more than 15 minutes — precisely the case the offline queue exists to serve. The client correctly treats the rejection as final, so the feed <i>and its photo</i> were discarded. The rule's own documentation says "a phone can be offline for hours."

AREA	DEFECT
Public collar page	The emergency button was disabled whenever the dog's profile failed to load — so a stranger on a poor connection, standing over an injured dog, saw a greyed-out "This dog needs help". Nothing in the emergency path needed that profile.
Authentication	Malformed tokens returned HTTP 500 with the internal error text, instead of 401. The guard intended to prevent this was unreachable code.
Login security	The 3-attempt cap on emailed login codes was a read-then-write race — concurrent attempts all read "0 used", so the cap never engaged against a 6-digit code.
Deployment	Re-deploys shipped nothing. The command used starts a service but does not restart an already-running one, so new code was built, "deploy complete" was printed, and the old code kept running. Health checks could not fail — every probe ended in <code> true</code> .
AI validator	Deleted a different job than the one it processed, so one scan was silently discarded while the processed job re-ran forever.
Photo capture	Two paths could wedge the primary action permanently: an unhandled module-load failure left a promise that never settled, and network requests had no timeout, so the SOS screen could display "Sending..." indefinitely.
Performance	Telemetry was 28% of the JavaScript on the critical path of the page whose entire design constraint is size. Now loaded after the page is usable.
Test integrity	Two tests failed on any modern Node version while CI's older version stayed green — and the failure was disguised as a data-loss bug. Separately, a component wrote to a browser global after being unmounted, which made an unrelated test suite fail intermittently.

8.3 · What was delivered

CHANGESET	CONTENTS	STATE
PR #17	Queue retry, offline skew clamp, both fail-open gates, auth error handling, login-cap race, timing-safe comparisons	MERGED & LIVE
PR #18	Unmounted-component global write; first QR frame decoded immediately rather than after a 350 ms delay	MERGED & LIVE
PR #19	Ledger ordering (<code>chain_seq</code>), ordering centralised into one module, migration 0017	GREEN, AWAITING MERGE

Net: 41 files changed, +1,917 / −221 lines. Test count 433 (2 failing) → 445 (all passing). Three database migrations. All seven gates passing.

8.4 · What remains open, and why

I did not fix everything, and the reasons differ in kind. Stating them precisely is more useful than a count.

OPEN ITEM	WHY IT IS STILL OPEN
Photo retention and AI validation jobs are never scheduled	Nothing enqueues them, so every scan sits at "pending" forever, badges can never be awarded, and the documented 7-day photo deletion is unimplemented — photos accumulate without bound. <i>Not fixed because</i> implementing it means choosing a permanent data-destruction policy, which is an owner's decision, not an auditor's.

OPEN ITEM	WHY IT IS STILL OPEN
No rate limiting anywhere	The endpoint that emails login codes is uncapped and sends mail synchronously. Roughly 300 unauthenticated requests would exhaust the email provider's free tier and break login for every user for the rest of the day. <i>Not fixed because</i> the project's own INVARIANT 6 forbids the obvious per-IP approach (Indian carriers use large-scale NAT, so per-IP limits lock out whole neighbourhoods), and choosing the real limits is a product decision.
Collar code check character is ineffective	It is supposed to catch a mistyped code before it becomes a lookup for the wrong dog. It catches neither of the two most common typo classes: the alphabet is 33 characters where the code assumes 32, and the checksum is order-independent so every adjacent transposition passes. <i>Not fixed because</i> I ran out of sequence, not because it is hard — this one is a genuine omission on my part.
Reverse-proxy config is never installed by automation	A server provisioned by following the project's own setup guide would serve none of the routing that the configuration describes, because nothing copies it into place. <i>Not fixed because</i> I cannot verify a change to the component that installs production routing without a server to test on.
A database password is committed to the repository	Present as a fallback default in four files, in a public repository. <i>Not fixed because</i> removing it from the code without rotating it on the server is theatre — the value is already in public git history. Rotation requires server access I do not have.
Ledger verification endpoint compares mismatched ranges	It recomputes over the first 1,000 records but compares against a daily anchor covering all of them, so it reports TAMPERED on an intact ledger once the ledger exceeds that size, or on the first record after any anchor. <i>Not fixed because</i> it is independent of the ordering bug and mixing two fixes in a ledger change destroys the ability to isolate a regression.

8.5 · A finding I rejected

An audit flagged that any authenticated user could hijack another user's push-notification registration, which would have been a serious access-control flaw. I implemented the fix, and an existing test failed — one whose fixtures were named "Original Owner" and "New Owner", indicating the behaviour was deliberate.

On review the existing behaviour is correct. A web-push endpoint identifies a *browser*, not a person: when a second user signs in on a shared phone, the browser returns the same endpoint. Blocking reassignment would mean the new user silently never receives emergency notifications on that device — a life-safety regression introduced in the name of security. I reverted my change.

WHY THIS BELONGS IN AN INVESTOR DOCUMENT

Because it is the difference between an audit and a checklist. The plausible-sounding fix would have degraded a life-safety path, and the thing that caught it was a test whose fixture *names* encoded the author's intent. That is a signal about the codebase's quality, and it is worth more than the defect count.

9 · Live production verification

Claims about a system are worth checking against the system. These were measured from the public internet on 15 August 2026, not read from documentation.

CHECK	RESULT	WHAT IT ESTABLISHES
GET <code>hetja.in/</code>	HTTP 200 · 18.4 KB · 1.29 s	The public site is live and serving.
GET <code>api.hetja.in/healthz</code>	HTTP 200 · 0.87 s	The API is up and reachable independently of the website.
GET <code>hetja.in/d/main.js</code>	HTTP 200 · 8,674 B gzipped	The collar page's code budget is real and being met in production.
GET <code>hetja.in/d/</code>	HTTP 200 · 3,268 B gzipped	Collar page total $\approx$ 11.9 KB against a self-imposed 40 KB ceiling.
Response headers on <code>/d/</code>	<code>cache-control: no-store</code>	The life-safety cache rule is enforced in production, not merely configured.
Deployment pipeline	Gate → Migrate → Deploy, all green	Deployed commit on the server matched the intended commit exactly; all four services reported active.

The 8,674-byte measurement is worth dwelling on, because it confirms something specific: the code-splitting change I made two hours earlier is live and delivering its benefit. The page a stranger loads while standing over an injured dog costs about **12 KB** — roughly one small photograph — against a budget of 40 KB. On a congested mobile network that is the difference between a page that renders and a page that does not.

WHAT THIS DOES NOT ESTABLISH

That the system is *used*. I could find no traction data of any kind: no user counts, no scan volume, no registered collars in production. The database I tested against contained 140 medical records, all of them test fixtures. "It is deployed and correct" and "it is adopted" are different claims, and only the first is supported by evidence available to me.

10 · Cost structure and the scaling ceiling

The system is engineered to run on almost no money, and it succeeds. The corollary is that the ceilings are close, and they are known.

10.1 · What it costs to run

The repository's recurring design filter is stated bluntly and repeatedly: "*this has to run on nothing*." The evidence supports it — one small virtual machine, a free Cloudflare plan, a free email tier, and a Supabase project holding schema only. Notably, cloud object storage was rejected **because it required a payment card on file** even for its free allowance.

10.2 · The four ceilings, in the order they will be hit

CEILING	MECHANISM	CONSEQUENCE
Disk	Dog photographs are written to the same disk as the database and served directly. The documented 7-day deletion policy is unimplemented — the routine that claims to delete old photos actually deletes rows from the job queue.	When the disk fills, PostgreSQL stops accepting writes and the emergency path stops working . This is stated in the repository.
Email quota	Login is by emailed code, on a free tier of 300 messages per day . There are no passwords and no alternative login path.	A hard cap on daily new users, and — with no rate limiting — a trivially triggered outage.
Memory	The four services' declared memory limits total roughly 1.68 GB before PostgreSQL starts , on a 2 GB machine that cannot swap and is shared with other workloads. The limits are annotated as tuned for a 3 GB machine — an unreconciled contradiction.	Already caused two outages, one of which produced no error message anywhere.
Table size	The highest-volume table is deliberately unpartitioned, documented as adequate to roughly 5 million rows per year.	Beyond that requires a migration that cannot be done with a single statement.

10.3 · The irreplaceable secret

The key signing collar QR codes exists in **one file, on one disk**, baked into every collar already glued to an animal. Losing it does not degrade the system — every collar stops verifying, and no code or schema recovers it. Regenerating it fails *silently*: discovered only when a stranger scans a collar while standing over a dog.

10.4 • The backup position

THIS IS THE MOST SERIOUS OPERATIONAL FINDING IN THE DOCUMENT

A backup routine exists, is well written, and runs nightly. **Its destination is a directory on the same disk it is backing up.** The repository says so itself and warns it "should not be counted as [an off-box backup] in any RTO estimate."

Point-in-time recovery tooling is installed but deliberately not enabled, and cannot work with the chosen storage provider — so the realistic recovery point is **up to 24 hours of data loss.**

No restore has ever been performed. The repository contains three separate instructions to carry one out, all unexecuted, including one that states: "*A backup you have never restored is a hypothesis.*"

11 · Risk register

Ordered by expected cost, not by likelihood. Each entry states the concrete failure, not a category.

RISK	CONCRETE FAILURE MODE	MITIGATION STATUS
Single server	Everything except the CDN runs on one 2 GB virtual machine. If it is lost, the register of every dog in the city is lost with it. A backup exists on paper; I found no evidence a restore has ever been performed.	UNMITIGATED
Untested restore	A backup that has never been restored is a hypothesis. One of the backup scripts also references a directory that does not exist, which can mark the whole nightly run as failed while logging almost nothing.	UNMITIGATED
Login denial of service	~300 unauthenticated requests exhaust the email provider's daily free tier; nobody can log in until the next day. No rate limiting exists anywhere in the API.	OPEN — \$8.4
Silent photo loss	If the storage backend is configured for S3, the code throws "not implemented" and the caller swallows it into a log line. The user sees success; no photo is ever stored.	IDENTIFIED, UNFIXED
Unbounded photo growth	The documented 7-day photo deletion is unimplemented and its job is never scheduled. Storage grows without limit on a 2 GB machine.	OPEN — \$8.4
Credential exposure	A database password is committed in a public repository and present in git history. Rotation requires server access.	OPEN — \$8.4
Key-person and provenance	Four days of history, 59 of 109 commits AI co-authored, and no evidence of a team beyond a small number of committer identities. Institutional knowledge lives in the repository's comments — which is unusually good, but it is not a team.	PARTLY MITIGATED BY DOCUMENTATION
Regulatory: right to erasure	The medical ledger is append-only by design and cannot delete records. India's DPDP Act contemplates erasure rights. These are in genuine tension; see §7.	UNRESOLVED

12 · What investment would actually buy

I have no budget, runway or valuation information, so I will not pretend to model any. What follows is what the *engineering evidence* says money would need to address, in priority order.

Tier 1 — Buys survival (weeks)

1. **Get off one machine, and prove the restore.** The single highest-expected-cost risk in this document is not a bug; it is that the entire register lives on one 2 GB server whose backup has never been restored. A second machine and one rehearsed restore changes the risk profile more than any feature.
2. **Rate limiting, then rotate the committed credential.** Both are small, both are currently exploitable by anyone who reads the public repository.
3. **Schedule the retention and validation jobs.** Until then, photos accumulate without bound on a machine with no headroom, and the AI review pipeline — which several user-facing features depend on — does nothing at all.

Tier 2 — Buys credibility (months)

1. **Close the ledger's remaining verification gap** so the public endpoint stops reporting tampering on intact data. The ledger is the differentiating claim; it should be airtight before it is marketed.
2. **Install the reverse-proxy configuration from automation** so a second city can be provisioned by following the guide, which is currently not true.
3. **Fix the collar check character.** Small, and it directly affects whether a mistyped code sends someone to the wrong dog.

Tier 3 — Buys the thesis

Everything above is remediation. The actual thesis — that a city-scale, open, auditable register of street animals is valuable — is not proven or disproven by any of it. What would prove it is adoption data, which does not yet exist. **The engineering is not the bottleneck; evidence of use is.**

THE INVESTMENT DECISION, STATED PLAINLY

On the evidence available, this is a bet on a team that can build unusually disciplined systems very fast, applied to a problem with real moral weight and no obvious revenue model in the repository. The code quality is genuinely above its age. The operational maturity is genuinely below what production demands. The gap between those two is small, well-understood, and fixable in weeks — which is the most encouraging thing I can say about it, and also the reason I would want to see adoption evidence before anything else.

Appendix A · Measured repository metrics

All figures measured directly on 15 August 2026 at commit `d8f8e1e` , excluding dependencies and build output.

COMPONENT	PURPOSE	LINES	RUNTIME DEPS
apps/web	Feeder app (Next.js)	6,422	11
apps/api	HTTP API (Fastify)	5,724	13
ops/	Deployment, gates, backups	3,063	—
packages/db	Schema, migrations, pool	2,769	—
apps/scan	Public collar page	1,584	2
packages/ledger	Hash chain, Merkle, signing	814	—
apps/worker	Job queue	764	4
packages/pow	Proof-of-work anti-abuse	321	—
packages/contracts	Shared validation schemas	249	—
apps/ai	Photo validation (Python)	162	—
Total source		21,872	
Total tests		9,484	

MEASURE	VALUE	NOTE
Tests passing	445	Across 9 packages; requires a real PostgreSQL with PostGIS + pgvector
Test-to-source ratio	0.43 : 1	High for a system of this age
Database tables	19	Across 20 migration files
API endpoints	26	All under <code>/api/v1/</code>
Automated gates	7	Security, contrast, cache policy, systemd wiring, query plans, destructive migrations, bundle size
Collar page budget	11.9 / 40 KB	Measured on the live site, gzipped
Commits	109	Over 4 calendar days; 59 AI co-authored

Appendix B · Glossary

TERM	MEANING IN THIS SYSTEM
Collar / slug	A physical QR tag on a dog. The "slug" is the 9-character code it encodes, signed so a forged code can be detected.
Scan	Any interaction recorded against a collar — a stranger viewing the page, or a feeder logging a meal.
Feeder	A registered volunteer who feeds and monitors specific dogs.
SOS case	An emergency report about an injured dog, which pages nearby responders and escalates if unacknowledged.
Hash chain	Records linked by cryptographic fingerprints, so altering an old record breaks every link after it.
Merkle root / anchor	A single fingerprint summarising every record, published daily so later alteration is detectable by comparison.
Append-only	Records can be added but never edited or deleted — corrections are new records that supersede old ones.
Advisory lock	A database mechanism ensuring only one writer at a time performs a sensitive operation.
Gate	An automated check that blocks a release when a rule is violated.
Migration	A versioned, ordered change to the database structure, applied automatically on deployment.
PostGIS / pgvector	PostgreSQL extensions for geographic queries and for AI image-similarity search respectively.
Progressive Web App	A website installable like an app, which works offline — chosen so users need no app store.
Fail open / fail closed	A failing check that lets bad input through (open) versus one that blocks it (closed). Gates must fail closed.

Provenance. Prepared by an AI engineering agent (Claude) that also contributed the three changesets described in §8.3. Findings were verified by reproduction against a running system wherever reproduction was possible. Figures marked "measured" were obtained by executing commands on 15 August 2026; all others are cited to repository files. No financial, market or team due diligence was performed, and none should be inferred from this document.